

Codebase & Architecture Guide

DUNDA — Event & Ticketing System

Project Documentation

DUNDA — Codebase & Architecture Guide (Educational)

A deep, teaching-oriented walkthrough of the DUNDA event & ticketing platform: **what it is**, **how the code is organized**, and **why each technology and architectural decision was made** — with real code snippets throughout.

Contents

1. [The project](#)
2. [The tech stack & why](#)
3. [Architecture](#)
4. [Codebase walkthrough \(with snippets\)](#)
5. [End-to-end trace: "Register for an event"](#)
6. [Concepts to take away](#)

1. The project

The problem

Discovering and organizing **free events** is fragmented. Attendees hunt across WhatsApp groups, Instagram posters, and word of mouth. Hosts stitch together a Google Form (RSVPs), a WhatsApp broadcast (promotion), and a spreadsheet (guest list). There's no single, trusted place that connects the two sides.

The solution

DUNDA is a **centralized platform** where:

- **Attendees** discover events (search + filters), RSVP, and get a **digital QR ticket**.
- **Hosts** create/manage events, assign a team (co-hosts, admins, volunteers), run **QR check-in**, and see **analytics**.
- **Organizations** (clubs, companies, universities) get a public page that people can **follow** to be notified of new events.

The actors (roles)

Role	Can do
Guest (not logged in)	Browse & filter events, view public event pages
Registered attendee	RSVP, get QR tickets, cancel, follow organizations
Host	Create/manage own events, assign team, check-in, analytics
Co-host / Admin / Volunteer	Per-event roles with decreasing privileges
Organization owner / admin	Manage an org page and its events

The domain model (plain English)

A **User** can host **Events**. Each event belongs to a **Category** and optionally an **Organization**. Users **register** for events (a **Registration**), and each registration gets a **Ticket** with a QR code. Events have contextual **EventRoles** (host/co-host/admin/volunteer). Private events use **EventInvites** (email allowlist) + an access code. Users **follow** organizations (**OrganizationFollow**) to get notified of new events.

2. The tech stack & why (each choice)

The guiding principle: **one cohesive, type-safe, full-stack codebase** that a small team can build and reason about quickly — not a sprawling microservice setup.

Next.js 16 (App Router) — the full-stack framework

Decision: Use Next.js for *both* frontend and backend, instead of a separate React SPA + an Express/Node API.

Why:

- **Server Components** render on the server and can query the database directly — no need to build and consume a REST API just to show a list of events.
- **Server Actions** let forms call server functions directly (with built-in CSRF protection), removing an entire layer of API endpoints, fetch calls, and request/response types.
- One deployment, one language, shared types end-to-end.

Trade-off: You're coupled to Next.js conventions. For this app that's a win; for a project needing a public API consumed by many clients, a separate API layer might be justified.

TypeScript (strict)

Decision: TypeScript everywhere, `strict: true`. **Why:** Catches whole classes of bugs at compile time, and — combined with Prisma and Zod — gives **end-to-end type safety** from the database row to the React prop. The build fails if types don't line up (`tsc --noEmit` is the gate).

Prisma 6 + SQLite — the data layer

Decision: Prisma ORM with a SQLite database in development. **Why Prisma:**

- You define the schema once; Prisma generates a **fully-typed client** and manages **migrations**.
- Queries are **parameterized**, which prevents SQL injection by construction.

Why SQLite (for a school project):

- Zero setup — the database is a single file (`prisma/dev.db`). Teammates clone and run.
- Portable: Prisma abstracts the SQL dialect, so moving to **PostgreSQL** for production is a one-line change (`provider = "postgresql"`).

Trade-off: SQLite is single-writer and file-based — great for dev, wrong for serverless/scale. That's an explicit, documented decision (see `docs/DEPLOYMENT.md`).

Note on SQLite + Prisma: SQLite doesn't support Prisma `enum`s, so enum-like fields (`status` , `role` , `visibility` , ...) are modelled as `String` with the allowed values documented in comments and enforced in application code.

Auth.js v5 (NextAuth) — authentication

Decision: Auth.js with a **Credentials** provider and **JWT** sessions. **Why:** It's the standard for Next.js auth, integrates with the App Router, and handles session cookies, CSRF, and the sign-in/out flow. JWT sessions avoid a database round-trip on every request. Passwords are hashed with **bcrypt**.

Zod + React Hook Form — validation

Decision: Define each form's rules once as a **Zod schema**, use it on the **client** (via React Hook Form) *and* re-run it on the **server**. **Why:** "Never trust the client." Client validation is for UX (instant feedback); server validation is for safety. Sharing one schema means the rules can't drift apart.

Tailwind CSS 4 + a hand-rolled UI kit

Decision: Tailwind for styling, with a small set of our own components (`Button` , `Input` , `Card` , ...) instead of a component library like MUI or shadcn's full install. **Why:** Full control over markup and styling, no heavy dependency, and it matches the app's exact look. Tailwind's utility classes keep styles co-located with markup.

The rest

- **UploadThing** — type-safe, Next.js-native **image uploads** with server-side auth gating.
- **Nodemailer + Ethereal** — transactional email; Ethereal is a fake inbox for dev so no real SMTP is needed to see emails.
- `qrcode` (generate) + `html5-qrcode` (scan) — digital tickets and camera check-in.

3. Architecture

3.1 The request lifecycle

DUNDA uses three server primitives, each for a specific job:

Primitive	Used for	Example
Server Component (default page)	Reading data & rendering HTML	The discovery page queries events and renders cards
Server Action (<code>"use server"</code>)	Mutations (create/update/delete)	Registering for an event
Route Handler (<code>route.ts</code>)	Files & machine endpoints	Ticket PNG download, <code>.ics</code> , the reminders cron

A page **reads** on the server and renders; a form **mutates** by calling a server action; a download is served by a route handler. There is **no separate REST API** in between.

3.2 Folder structure (route groups)

```

app/
  (public)/           # pages anyone can see (grouped for a shared layout)
    page.tsx         # landing
    events/          # discovery, details, register, manage, check-in, analytics
    orgs/             # org discovery, org page, org manage
  (auth)/             # login, signup (centered layout, no navbar)
  (app)/              # logged-in-only pages (account, onboarding, create)
  api/               # route handlers (auth, uploadthing, tickets, ics, cron)
  layout.tsx         # root layout + site metadata
components/          # UI kit + feature components (navbar, event-card, forms)
lib/                 # db client, auth config, validators, domain helpers, email, qr, ics
prisma/              # schema.prisma + seed.ts + migrations

```

`(public)`, `(auth)`, `(app)` are **route groups** — the parentheses mean the folder name does *not* appear in the URL; it only groups routes that share a layout. (A subtle rule learned the hard way: the same dynamic segment like `events/[id]` must live in **one** group, or Next can't resolve it.)

3.3 Authentication: an edge-safe split config

Auth config is split into two files so route protection can run in the **Edge runtime** (which can't use Node APIs like Prisma), while the actual password check runs in **Node**.

`auth.config.ts` (edge-safe — no database imports) holds the callbacks, including route protection:

```
// auth.config.ts
export const authConfig = {
  pages: { signIn: "/login" },
  session: { strategy: "jwt" },
  providers: [], // the real provider is added in auth.ts
  callbacks: {
    authorized({ auth, request: { nextUrl } }) {
      const isLoggedIn = !!auth?.user;
      const path = nextUrl.pathname;
      const needsAuth =
        path.startsWith("/account") ||
        path === "/events/new" ||
        /^\/events\/[^\s]+\/(manage|check-in|register|analytics)/.test(path) ||
        /^\/orgs\/[^\s]+\/manage/.test(path);
      if (needsAuth && !isLoggedIn) return false; // → redirect to /login
      return true;
    },
    jwt({ token, user }) { if (user) token.id = user.id as string; return token; },
    session({ session, token }) { if (token.id) session.user.id = token.id as string; return session; },
  },
} satisfies NextAuthConfig;
```

`auth.ts` (Node — imports Prisma + bcrypt) adds the Credentials provider:

```
// auth.ts
export const { handlers, auth, signIn, signOut } = NextAuth({
  ...authConfig,
  providers: [
    Credentials({
      authorize: async (credentials) => {
        const parsed = loginSchema.safeParse(credentials);
        if (!parsed.success) return null;
        const user = await prisma.user.findUnique({ where: { email: parsed.data.email } });
        if (!user) return null;
        const valid = await bcrypt.compare(parsed.data.password, user.passwordHash);
        return valid ? { id: user.id, email: user.email, name: user.name } : null;
      },
    }),
  ],
});
```

`proxy.ts` (Next 16's renamed middleware) wires the edge config to every request:

```
// proxy.ts
import NextAuth from "next-auth";
import { authConfig } from "@/auth.config";
export default NextAuth(authConfig).auth; // enforces `authorized()` on each request
export const config = { matcher: ["/((?!api|_next/static|_next/image|favicon.ico|.*\\.svg$).*)" ] };
```

3.4 The data layer: one Prisma client

Prisma is instantiated once and reused across hot-reloads (otherwise dev would exhaust DB connections):

```
// lib/db.ts
const globalForPrisma = globalThis as unknown as { prisma: PrismaClient | undefined };
export const prisma = globalForPrisma.prisma ?? new PrismaClient();
if (process.env.NODE_ENV !== "production") globalForPrisma.prisma = prisma;
```

3.5 Validation strategy (shared Zod schemas)

Every form has one schema in `lib/validators/`. Example (abridged) event schema with a cross-field rule:

```
// lib/validators/event.ts
export const eventSchema = z
  .object({
    title: z.string().min(3).max(120),
    format: z.enum(["IN_PERSON", "VIRTUAL"]),
    venue: z.string().optional().or(z.literal("")),
    startAt: z.string().min(1, "Start date and time is required"),
    registrationDeadline: z.string().optional().or(z.literal("")),
    // ...capacity, visibility, recurrence, images...
  })
  .superRefine((data, ctx) => {
    if (data.format === "IN_PERSON" && !data.venue)
      ctx.addIssue({ path: ["venue"], code: "custom", message: "Venue is required for in-person events" });
    if (new Date(data.startAt).getTime() < Date.now())
      ctx.addIssue({ path: ["startAt"], code: "custom", message: "Event date must be in the future" });
  });
export type EventInput = z.infer<typeof eventSchema>; // the form's TypeScript type, for free
```

The same schema validates in the browser (React Hook Form) *and* inside the server action.

3.6 Authorization model (contextual roles)

Roles are **per-event**, not a single global column. An `EventRole` row ties a user to an event with a role; helper functions centralize "who can do what":

```
// lib/events.ts
export const CAN_EDIT_ROLES = ["HOST", "COHOST", "ADMIN"];
export const CAN_DELETE_ROLES = ["HOST"];
export const CAN_CHECKIN_ROLES = ["HOST", "COHOST", "ADMIN", "VOLUNTEER"];

export async function getUserEventRole(userId: string | undefined, eventId: string) {
  if (!userId) return null;
  const role = await prisma.eventRole.findUnique({
    where: { eventId_userId: { eventId, userId } },
    select: { role: true },
  });
  return role?.role ?? null;
}
```

Every sensitive page and action calls `getUserEventRole` and checks it against the right constant.

3.7 Business rules as small pure functions

Time-based rules live in one place and are reused by the UI *and* the server action — so the button and the backend can never disagree:

```
// lib/events.ts
// Registration closes at the EARLIER of the deadline or the start time.
export function registrationCloseTime(e: { startAt: Date; registrationDeadline: Date | null }) {
  return e.registrationDeadline && e.registrationDeadline < e.startAt ? e.registrationDeadline : e.startAt
}
export const isRegistrationOpen = (e: { startAt: Date; registrationDeadline: Date | null }) =>
  registrationCloseTime(e).getTime() > Date.now();

// Once an event has started, its details are locked from editing.
export const hasEventStarted = (startAt: Date) => startAt.getTime() <= Date.now();
```

4. Codebase walkthrough

4.1 The schema (`prisma/schema.prisma`)

The heart of the app. A few representative models:


```

model User {
  id          String   @id @default(cuid())
  email       String   @unique
  passwordHash String           // bcrypt hash, never plaintext
  name        String
  username    String   @unique
  hostedEvents Event[] @relation("EventHost")
  registrations Registration[]
  // ...interests, orgs, follows...
}

model Event {
  id          String   @id @default(cuid())
  title       String
  startAt     DateTime
  registrationDeadline DateTime? // optional cutoff (Sprint 9)
  format      String    // IN_PERSON | VIRTUAL
  capacity    Int?
  visibility  String    @default("PUBLIC") // PUBLIC | PRIVATE
  accessCode  String?   // for private events
  hostId      String
  organizationId String?
  host        User      @relation("EventHost", fields: [hostId], references: [id])
  organization Organization? @relation(fields: [organizationId], references: [id])
  roles       EventRole[]
  registrations Registration[]
}

model Registration {
  id          String @id @default(cuid())
  eventId     String
  userId      String
  status      String @default("CONFIRMED") // CONFIRMED | CANCELLED
  ticket      Ticket?
  @@unique([eventId, userId])             // ← you can't register twice
}

model Ticket {
  id          String @id @default(cuid())
  registrationId String @unique
  code        String @unique @default(cuid()) // encoded in the QR
  status      String @default("VALID")       // VALID | USED | INVALID
}

```

Notice how much *correctness* is expressed here: `@unique([eventId, userId])` makes double registration impossible at the database level; the one-to-one `Ticket.registrationId @unique` means one ticket per registration.

4.2 A page that reads data (Server Component)

The discovery page (`app/(public)/events/page.tsx` , abridged) builds a Prisma `where` from the URL's search params and renders — all on the server, no API call:

```
export default async function DiscoverPage({ searchParams }: { searchParams: Promise<Record<string, string> } }) {
  const sp = await searchParams;
  const where: Prisma.EventWhereInput = {
    status: "PUBLISHED", visibility: "PUBLIC",
    startAt: { gte: new Date() }, // upcoming only
  };
  if (sp.q) where.OR = [{ title: { contains: sp.q } }, { description: { contains: sp.q } }];
  if (sp.category) where.categoryId = sp.category;
  if (sp.format) where.format = sp.format;

  const events = await prisma.event.findMany({ where, include: { category: true }, orderBy: { startAt: "asc" });
  return <div className="grid gap-5 sm:grid-cols-3">{events.map(e => <EventCard key={e.id} event={e} />)}
}
```

4.3 The UI kit (components/ui/)

Small, typed wrappers around HTML elements with Tailwind classes and variants — no external component library:

```
// components/ui/button.tsx (abridged)
const variants = {
  primary: "bg-primary text-primary-foreground hover:bg-primary-hover",
  outline: "border border-border bg-surface hover:bg-background",
  danger: "bg-danger text-white hover:bg-red-700",
};
export function Button({ variant = "primary", size = "md", className, ...props }: ButtonProps) {
  return <button className={cn(base, variants[variant], sizes[size], className)} {...props} />;
}
```

`cn()` merges Tailwind classes safely (clsx + tailwind-merge).

4.4 Image uploads (UploadThing)

The **file route** is defined server-side and **gated by session** — only logged-in users can upload:

```
// app/api/uploadthing/core.ts
export const ourFileRouter = {
  imageUploader: f({ image: { maxFileSize: "4MB", maxFileCount: 1 } })
    .middleware(async () => {
      const session = await auth();
      if (!session?.user?.id) throw new UploadThingError("You must be logged in to upload.");
      return { userId: session.user.id };
    })
    .onUploadComplete(async ({ file }) => ({ url: file.ufsUrl })),
} satisfies FileRouter;
```

The reusable `<ImageUpload>` component uploads a file and writes the returned URL back into the form field — with a graceful fallback to a plain URL text box when uploads aren't configured.

5. End-to-end trace: "Register for an event"

This single feature touches every layer — a great way to see how the pieces connect.

Step 1 — The page (read + gate). `app/(public)/events/[id]/register/page.tsx` runs on the server, checks auth, checks access (private events), and checks the registration window *before* showing the form:

```
const session = await auth();
if (!session?.user?.id) redirect(`/login?callbackUrl=/events/${id}/register`);
if (!(await hasEventAccess({ event, userId: session.user.id, code }))) redirect(`/events/${id}`);
const registrationClosed = !isRegistrationOpen(event); // uses the shared rule
```

Step 2 — The client form. React Hook Form validates with the **shared Zod schema** for instant feedback, then calls the server action:

```
const onSubmit = handleSubmit((data) => startTransition(async () => {
  const result = await registerForEventAction(eventId, data); // ← calls the server
  if (result?.error) setFormError(result.error);
}));
```

Step 3 — The server action (the important part). `registerForEventAction` **re-validates**, re-checks the window, then does the capacity + duplicate guard **inside a transaction** so two people racing for the last seat can't both win:

```

"use server";
export async function registerForEventAction(eventId: string, input: RegistrationInput) {
  const session = await auth();
  if (!session?.user?.id) redirect("/login");

  const parsed = registrationSchema.safeParse(input); // server-side validation
  if (!parsed.success) return { errors: parsed.error.flatten().fieldErrors };

  const event = await prisma.event.findUnique({ where: { id: eventId } });
  if (!event || event.status !== "PUBLISHED") return { error: "This event is not available." };
  if (!isRegistrationOpen(event)) return { error: "Registration for this event has closed." };

  try {
    await prisma.$transaction(async (tx) => {
      const existing = await tx.registration.findUnique({ where: { eventId_userId: { eventId, userId: session.user.id } } });
      if (existing?.status === "CONFIRMED") throw new RegistrationError("You are already registered.");
      if (event.capacity !== null) {
        const count = await tx.registration.count({ where: { eventId, status: "CONFIRMED" } });
        if (count >= event.capacity) throw new RegistrationError("This event is fully booked.");
      }
      const reg = await tx.registration.create({ data: { eventId, userId: session.user.id, name, email } });
      await tx.ticket.create({ data: { registrationId: reg.id } }); // ticket + QR code
    });
  } catch (e) {
    if (e instanceof RegistrationError) return { error: e.message };
    throw e;
  }

  await sendEmail({ to: email, ...registrationConfirmationEmail(name, event) }); // confirmation email
  revalidatePath(`/events/${eventId}`); // refresh cached page
  redirect("/account/tickets"); // → My Tickets
}

```

Step 4 — The ticket & QR. The ticket's `code` is encoded into a QR image on the *My Tickets* page. Scanning it opens the check-in page with `?code=...`, and the check-in action flips the ticket `VALID` → `USED`.

What this one feature demonstrates:

- **Defense in depth** — validated on the client (UX) *and* the server (safety).
- **Data integrity** — capacity/duplicate handled in a DB transaction, not with racy `if` checks.
- **Separation of concerns** — page reads & gates, action mutates, helper functions hold the rules, email is its own module.

6. Concepts to take away

These are the transferable lessons — useful well beyond this project:

1. **Server Components vs Client Components.** Pages are server components by default (they can touch the DB and render HTML). Add `"use client"` only where you need interactivity (state, event handlers). This keeps most code on the server and ships less JavaScript.

2. **Server Actions** collapse the traditional "build an API + fetch it" into a single typed function call — with CSRF protection built in.
3. **Type safety end-to-end.** Prisma types the database, Zod types the inputs (`z.infer`), and TypeScript ties it together. A wrong field name fails the build, not production.
4. **Validate twice.** Client validation is UX; server validation is security. Share one schema so they never drift.
5. **Authorize on the server, every time.** Hiding a button in the UI is not security — the server action re-checks the user's role (`getUserEventRole`) before mutating.
6. **Encode invariants in the database.** `@@unique([eventId, userId])` makes an entire class of bug (double registration) impossible, no matter what the application code does.
7. **Transactions for correctness under concurrency.** Counting seats and inserting must be atomic, or two users can overbook the last spot.
8. **Pure functions for business rules.** `isRegistrationOpen` , `hasEventStarted` , and the role constants are tiny, testable, and shared — so the UI and backend always agree.
9. **Migrations, not manual SQL.** The schema is the source of truth; `prisma migrate` evolves the database in versioned, reviewable steps.
10. **Choose boring, portable defaults.** SQLite + Prisma runs anywhere and swaps to Postgres with one line — deferring an operational decision until it actually matters.

See also: `docs/AUDIT.md` (code review & findings), `docs/DEPLOYMENT.md` (setup & hosting), `docs/evaluation-instrument.md` (user-testing instrument).